Devixo Solutions

AI/ML Internship Program

Task 03

End-to-End Machine Learning Project

Project: Loan Default Prediction

Student Name: Aman Hameed Ullah

Internship Domain: AI/ML

Submission Date: 08-11-2026

Table of Contents

SECTION 1: Introduction 3
SECTION 2: Objective 3
SECTION 3: Dataset Description 4
SECTION 4: Methodology 4
SECTION 5: Data Preparation 5
SECTION 6: Model Development 8
SECTION 7: Model Optimization 10
SECTION 8: Saving the Best Model 15
SECTION 9: Prediction Interface with Streamlit (Bonus) 16
SECTION 10: Model Selection and Results 18
SECTION 11: Limitations and Future Improvements 19
SECTION 12: Conclusion 19
SECTION 13: GitHub Repository Link 20

SECTION 1: Introduction

When a bank or lender grants a loan, there is always a risk that the borrower will not repay it (loan default). Every default means a financial loss, while rejecting a good borrower means lost business. A model that estimates the default risk from the information in the loan application helps lenders to decide faster and more consistently.

This report documents Task 03 of the Devixo Solutions AI/ML Internship Program: an end-to-end machine learning project on the Credit Risk dataset from Kaggle. The project covers data preparation with feature engineering and feature selection, the development of five models, their optimization with cross-validation, hyperparameter tuning and feature importance analysis, saving the best model with Joblib and Pickle, and a Streamlit interface for predictions. The code is organized as a reusable scikit-learn pipeline, so that new applicants can be scored in one step.

SECTION 2: Objective

The objectives of this project are to:

- Select a real-world Kaggle dataset with at least 3,000 rows and prepare it (missing values, duplicates, feature engineering, feature selection and scaling).
- Train at least four different machine learning models.
- Optimize the models with hyperparameter tuning, cross-validation and feature importance analysis.
- Save the best-performing model with Joblib and Pickle.
- Document the dataset, workflow, model selection, results, limitations and future improvements.
- Build a simple prediction interface with Streamlit (bonus).

Table 1: Task requirements and where they are covered

Task Requirement	Covered in
Part 1: Data preparation (missing values, duplicates, feature engineering, feature selection, scaling)	Section 5
Part 2: Model development (Decision Tree, Random Forest, Gradient Boosting, SVM, XGBoost)	Section 6
Part 3: Model optimization (hyperparameter tuning, cross-validation, feature importance)	Section 7
Part 4: Saving the best model (Joblib and Pickle)	Section 8
Part 5: Documentation (dataset, workflow, model selection, results, limitations, future improvements)	Sections 3, 4, 10 and 11
Bonus: Prediction interface with Streamlit	Section 9

SECTION 3: Dataset Description

Table 2: Dataset details

Item	Detail
Dataset Name	Credit Risk Dataset
Source	Kaggle: https://www.kaggle.com/datasets/laotse/credit-risk-dataset
Creator	Lao Tse
Size (raw)	32,581 rows × 12 columns
Size (after cleaning)	32,409 rows × 16 columns (including 4 engineered features)
File	credit_risk_dataset.csv
Target Column	loan_status (1 = default, 0 = repaid); about 21.8% of the loans defaulted
Problem Type	Binary classification

Each row is one loan applicant. The dataset contains numerical and categorical columns, missing values, duplicate rows and impossible values, which makes it a realistic example for a complete machine learning workflow. The columns are grouped in Table 3.

Table 3: Column groups of the dataset

Group	Columns	Description
Borrower	person_age, person_income, person_home_ownership, person_emp_length	Age, yearly income, housing situation and years of employment
Loan	loan_intent, loan_grade, loan_amnt, loan_int_rate, loan_percent_income	Purpose, risk grade (A to G), amount, interest rate and loan as a share of income
Credit history	cb_person_default_on_file, cb_person_cred_hist_length	Earlier default and length of the credit history
Target	loan_status	1 if the loan defaulted, otherwise 0

SECTION 4: Methodology

The project followed this workflow:

Data Preparation → Reusable Pipeline → Model Development → Model Optimization → Saving the Best Model → Documentation → Streamlit Interface

Tools and technologies used:

- Programming language: Python 3
- Environment: Google Colab (Jupyter notebook) and VS Code (Streamlit interface)

- Libraries: Pandas and NumPy (data handling), Matplotlib and Seaborn (charts), Scikit-learn (pipeline, models and metrics), XGBoost, Joblib and Pickle (saving the model), Streamlit (interface)
- Storage and version control: Google Drive and GitHub

SECTION 5: Data Preparation

5.1 Loading the Data and Understanding the Target

The dataset was loaded from Google Drive with pd.read_csv(). It has 32,581 rows and 12 columns. About 78.18% of the loans were repaid and 21.82% ended in default, so the classes are imbalanced: a model that always predicts "no default" would already reach 78.18% accuracy. For this reason precision, recall and F1 score are used together with accuracy.

```
display(df.head())
df.info()
```

```
Shape: (32581, 12)
```

	person_age	person_income	person_home_ownership	person_emp_length	loan_intent	loan_grade	loan_amnt	loan_int_rate	loan_status	loan_percent_income	cb_person_default_on_file	cb_person_cred_hist_length
0	22	59000	RENT	123.0	PERSONAL	D	35000	16.02	1	0.59	Y	3
1	21	9600	OWN	5.0	EDUCATION	B	1000	11.14	0	0.10	N	2
2	25	9600	MORTGAGE	1.0	MEDICAL	C	5500	12.87	1	0.57	N	3
3	23	65500	RENT	4.0	MEDICAL	C	35000	15.23	1	0.53	N	2
4	24	54400	RENT	8.0	MEDICAL	C	35000	14.27	1	0.55	Y	4

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32581 entries, 0 to 32580
Data columns (total 12 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   person_age                  32581 non-null  int64
 1   person_income               32581 non-null  int64
 2   person_home_ownership       32581 non-null  object
 3   person_emp_length           31686 non-null  float64
 4   loan_intent                 32581 non-null  object
 5   loan_grade                  32581 non-null  object
 6   loan_amnt                   32581 non-null  int64
 7   loan_int_rate               29465 non-null  float64
 8   loan_status                 32581 non-null  int64
 9   loan_percent_income         32581 non-null  float64
 10  cb_person_default_on_file   32581 non-null  object
 11  cb_person_cred_hist_length  32581 non-null  int64
dtypes: float64(3), int64(5), object(4)
memory usage: 3.0+ MB
```

Figure 1: Dataset records and information

```
loan_status
0    25473
1     7108
Name: count, dtype: int64
loan_status
0    78.18
1    21.82
Name: proportion, dtype: float64
```

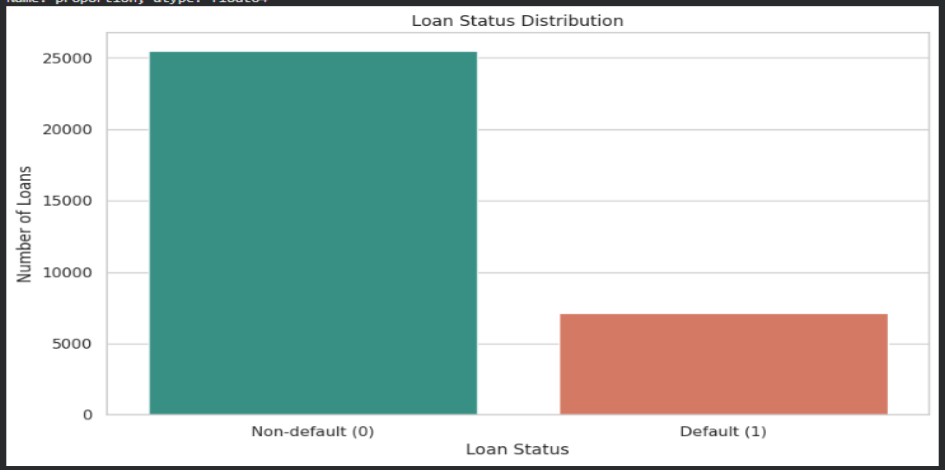


Figure 2: Distribution of the target variable (loan_status)

5.2 Missing Values, Duplicates and Invalid Values

Two columns contain missing values: person_emp_length (895 values, 2.75%) and loan_int_rate (3,116 values, 9.56%). They are filled with the median inside the pipeline (Section 5.5) and not before the split, so that the median is learned from the training data only. The median is used because both columns are skewed.

The dataset contained 165 duplicate rows, which were removed because duplicates give extra weight to the same record and could appear in both the training and the test set. The summary statistics also showed impossible values: 5 rows with an age above 100 (up to 144 years) and 2 rows with an employment length above 60 years (123 years). These 7 rows were removed. Very high incomes were kept because they are possible. After cleaning, 32,409 rows remained.

	Missing	Percent
person_emp_length	895	2.75
loan_int_rate	3116	9.56

	count	mean	std	min	25%	50%	75%	max
person_age	32581.0	27.734600	6.348078	20.00	23.00	26.00	30.00	144.00
person_income	32581.0	66074.848470	61983.119168	4000.00	38500.00	55000.00	79200.00	6000000.00
person_emp_length	31686.0	4.789686	4.142630	0.00	2.00	4.00	7.00	123.00
loan_amnt	32581.0	9589.371106	6322.086646	500.00	5000.00	8000.00	12200.00	35000.00
loan_int_rate	29465.0	11.011695	3.240459	5.42	7.90	10.99	13.47	23.22
loan_status	32581.0	0.218164	0.413006	0.00	0.00	0.00	0.00	1.00
loan_percent_income	32581.0	0.170203	0.106782	0.00	0.09	0.15	0.23	0.83
cb_person_cred_hist_length	32581.0	5.804211	4.055001	2.00	3.00	4.00	8.00	30.00

Figure 3: Missing values and summary statistics

5.3 Feature Engineering

Four new features were created from existing columns (Table 4). They combine information that is spread over several columns and may help the models to detect risky borrowers. They are calculated row by row, so creating them before the split does not leak information.

Table 4: Engineered features

Feature	Definition	Reason
emp_to_age_ratio	Employment length divided by age	Measure of work stability
interest_cost	Loan amount × interest rate / 100	Real yearly cost of the loan
interest_to_income	interest_cost divided by income	How heavily the interest burdens the borrower
age_group	Age in four bands (up to 25, 26-35, 36-45, 46+)	Lets models find differences between age groups

5.4 Train-Test Split

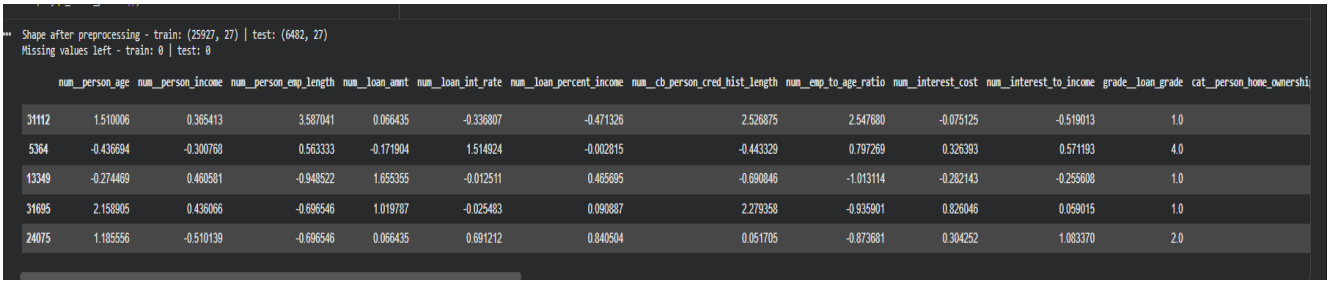
The data was split into 80% for training (25,927 loans) and 20% for testing (6,482 loans). stratify=y keeps the default rate at about 21.9% in both sets, and random_state=42 makes the split reproducible. The split was done before imputing, scaling and feature selection, so that these steps learn from the training data only and the test set stays completely unseen.

5.5 Reusable Preprocessing Pipeline

All preprocessing was combined in one scikit-learn ColumnTransformer, which treats each group of columns in the right way (Table 5). After preprocessing there are 27 numeric features and no missing values. Scaling is important for SVM, whose result depends on distances, while tree-based models are not affected by it. The pipeline was fitted only on the training data and then applied to the test data.

Table 5: Preprocessing steps in the pipeline

Column Group	Columns	Treatment
Numerical (10)	person_age, person_income, person_emp_length, loan_amnt, loan_int_rate, loan_percent_income, cb_person_cred_hist_length and the 3 engineered numerical features	Median imputation, then standardization (mean 0, standard deviation 1)
Ordinal (1)	loan_grade	Ordinal encoding in the order A < B < C < D < E < F < G
Nominal (4)	person_home_ownership, loan_intent, cb_person_default_on_file, age_group	One-hot encoding (handle_unknown = ignore)



```
Shape after preprocessing - train: (25927, 27) | test: (6482, 27)
Missing values left - train: 0 | test: 0
```

	num__person_age	num__person_income	num__person_emp_length	num__loan_amnt	num__loan_int_rate	num__loan_percent_income	num__cb_person_cred_hist_length	num__emp_to_age_ratio	num__interest_cost	num__interest_to_income	grade__loan_grade	cat__person_home_ownershi
31112	1.510006	0.365413	3.587041	0.066435	-0.336807	-0.471326	2.526875	2.547680	-0.075125	-0.519013	1.0	
5364	-0.436694	-0.300768	0.563333	-0.171904	1.514924	-0.002815	-0.443329	0.797269	0.326393	0.571193	4.0	
13349	-0.274469	0.460581	-0.948522	1.655355	-0.012511	0.465695	-0.690846	-1.013114	-0.282143	-0.255608	1.0	
31695	2.158905	0.436066	-0.696546	1.019787	-0.025483	0.090887	2.279358	-0.935901	0.826046	0.059015	1.0	
24075	1.185556	-0.510139	-0.696546	0.066435	0.691212	0.840504	0.051705	-0.873681	0.304252	1.083370	2.0	

Figure 4: Preprocessing pipeline and its output

5.6 Feature Selection

Feature selection removes features that carry little information, which makes the models simpler and faster. The ANOVA F-test measures how strongly each feature is related to the target, and SelectFpr keeps only the features whose relationship with the target is statistically significant (p-value below 0.05). The test was calculated on the training data only. 25 of the 27 features were kept; the two removed features are age_group_36-45 and age_group_46+. The strongest features are interest_to_income (F-score 6,140), loan_percent_income (4,373), loan_grade (4,186) and loan_int_rate (2,939). The engineered feature interest_to_income is the strongest feature of all,

which shows that feature engineering added useful information. The selector is part of the final pipeline, so it is applied automatically to every new applicant.

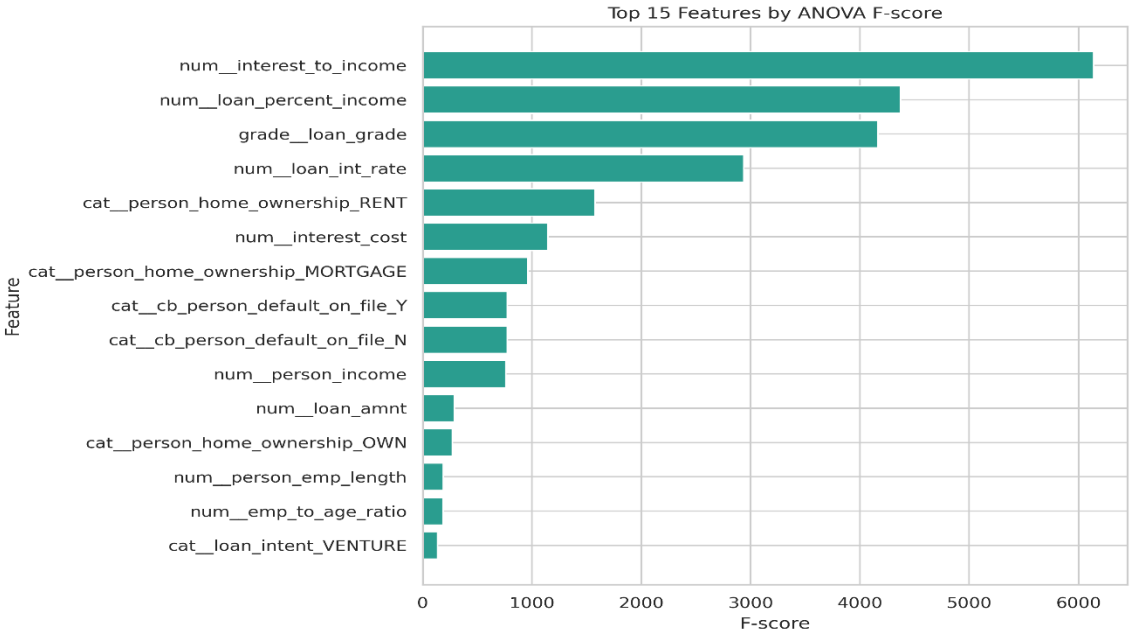


Figure 5: Top 15 features by ANOVA F-score

SECTION 6: Model Development

Five classification models from different algorithm families were trained. Each one is a complete scikit-learn Pipeline (preprocessing, feature selection and model), so every model receives raw data and there is no risk of data leakage.

Table 6: Models used

Model	Family	How it works	Tuned later
Decision Tree	Tree	Splits the data with yes/no questions until it reaches a prediction (no depth limit)	Yes
Random Forest	Ensemble (bagging)	Combines the votes of 200 trees trained on random samples of rows and features	Yes
Gradient Boosting	Boosting	Builds shallow trees one after another, each correcting the errors of the previous ones	Yes
SVM	Kernel-based	Finds the boundary with the largest margin between the classes (RBF kernel)	No (slowest)
XGBoost	Boosting (optional)	Fast, regularized implementation of gradient boosting (200 trees, depth 4)	Yes

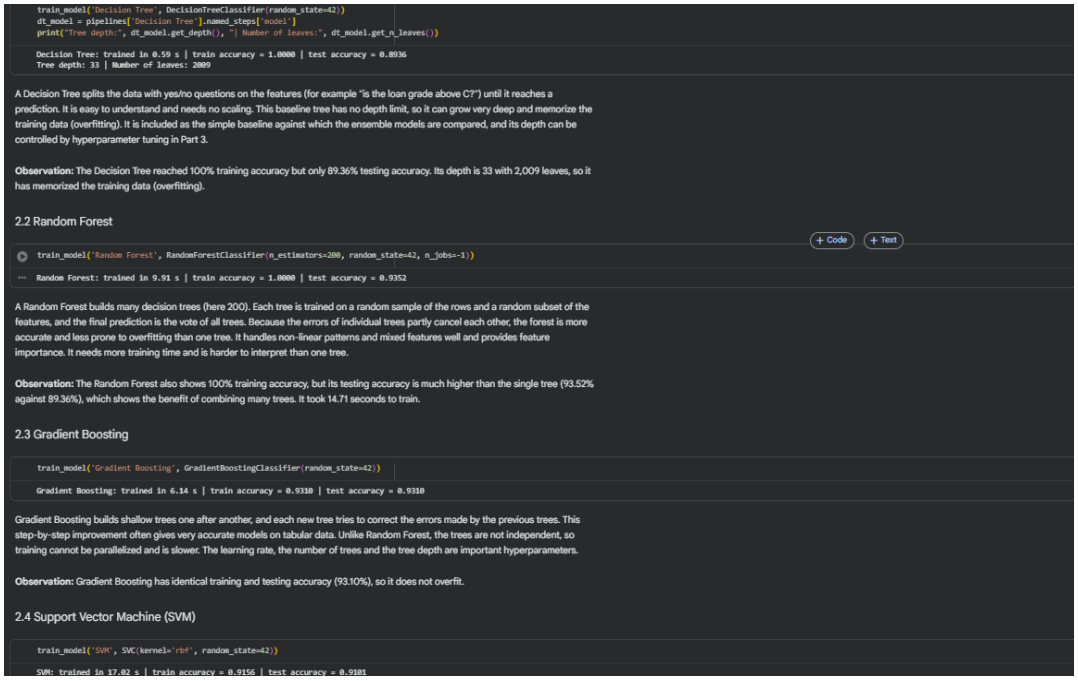


Figure 6: Code and output of model training

The models were evaluated on the test set of 6,482 loans. The positive class is "default". Precision tells how many of the loans predicted to default really defaulted, and recall tells how many of the real defaults were found. For a lender, a missed default is usually more expensive than a rejected good loan, so recall and F1 are as important as accuracy.

Table 7: Results of the models with default settings (test set)

Model	Time (s)	Train Acc.	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost	2.24	0.9387	0.9397	0.9849	0.7355	0.8421	0.9499
Random Forest	14.71	1.0000	0.9352	0.9699	0.7264	0.8306	0.9367
Gradient Boosting	6.29	0.9310	0.9310	0.9500	0.7228	0.8210	0.9325
Decision Tree	0.63	1.0000	0.8936	0.7398	0.7920	0.7650	0.8570
SVM	18.67	0.9156	0.9101	0.9196	0.6453	0.7584	0.8937

XGBoost is the best model on accuracy (93.97%), precision, F1 score (0.8421) and ROC-AUC (0.9499), and it trains in only 2.24 seconds. The Decision Tree (depth 33, 2,009 leaves) and the Random Forest reach 100% accuracy on the training data but less on the test data, so they overfit; XGBoost and Gradient Boosting have almost identical training and testing accuracy. The Decision Tree has the highest recall (79.20%) but a low precision (73.98%), so it also flags many good loans. SVM is the slowest model and has the lowest recall (64.53%). All models are clearly better than the 78.18% baseline.

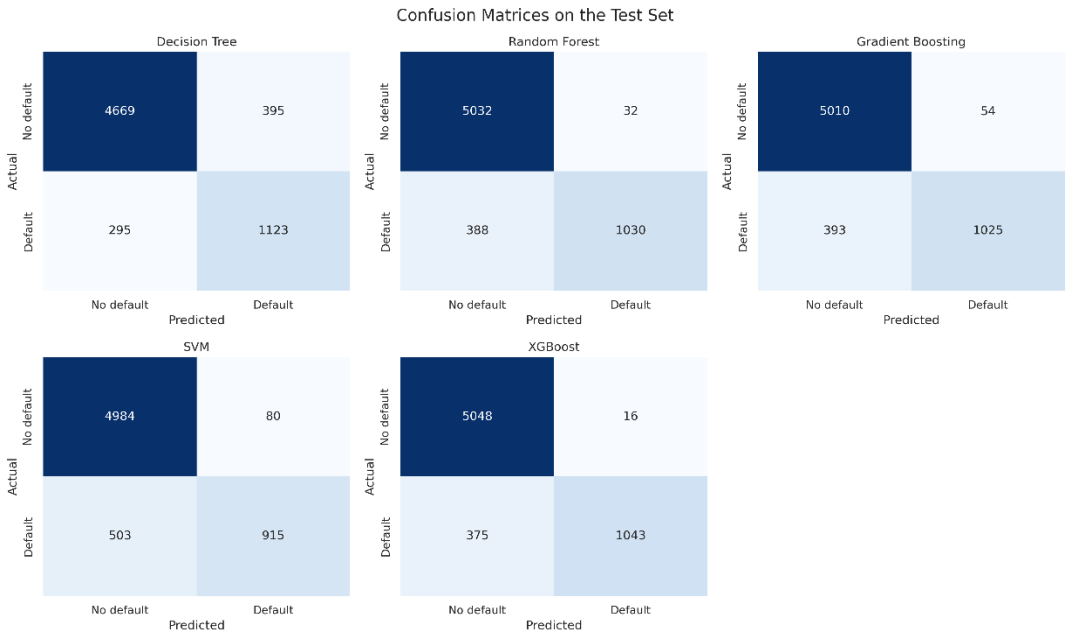


Figure 7: Confusion matrices of the five models on the test set

Table 8: Confusion matrix counts on the test set

Model	TN	FP	FN	TP
XGBoost	5048	16	375	1043
Random Forest	5032	32	388	1030
Gradient Boosting	5010	54	393	1025
Decision Tree	4669	395	295	1123
SVM	4984	80	503	915

The test set contains 1,418 real defaults. XGBoost finds 1,043 of them and raises only 16 false alarms, but it still misses 375. The Decision Tree finds the most defaults (1,123) but raises 395 false alarms, and SVM misses the most (503). The ensemble models have a very high precision (95% to 98%) but a recall of only 65% to 74%, which is the main weakness that the optimization tries to improve.

SECTION 7: Model Optimization

7.1 Cross-Validation

A single train-test split can be lucky or unlucky. In 5-fold stratified cross-validation the training data is divided into 5 parts; each part is used once for validation while the model is trained on the other four. The whole pipeline is refitted inside every fold, so there is no data leakage.

Table 9: 5-fold cross-validation results (training data)

Model	Accuracy	Recall	F1	F1 Std	ROC-AUC
XGBoost	0.9352	0.7233	0.8299	0.0040	0.9431
Random Forest	0.9327	0.7168	0.8233	0.0050	0.9305
Gradient Boosting	0.9297	0.7145	0.8164	0.0043	0.9287
SVM	0.9094	0.6423	0.7562	0.0064	0.8911
Decision Tree	0.8905	0.7677	0.7542	0.0097	0.8463

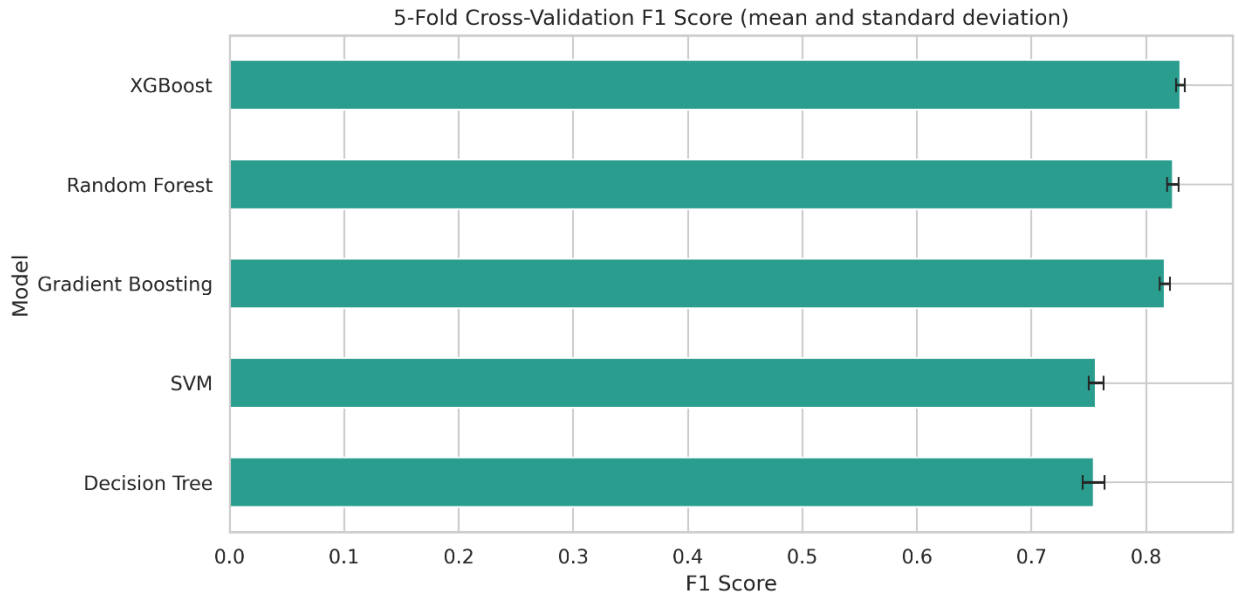


Figure 8: Cross-validation F1 score (mean and standard deviation)

The ranking agrees with the test results of Section 6. The standard deviation of the F1 score is small for all models (0.0040 to 0.0097), so the results are stable and do not depend on one lucky split. XGBoost is the best and the most stable model, and it needed only 3.5 seconds for the cross-validation, compared with 89.9 seconds for SVM.

7.2 Hyperparameter Tuning

Hyperparameters are settings chosen before training, such as the tree depth or the learning rate. GridSearchCV tries every combination of a small grid (Decision Tree), and RandomizedSearchCV tries a fixed number of random combinations from a large space (Random Forest, Gradient Boosting and XGBoost). Each combination was evaluated with 3-fold stratified cross-validation on the training data only, using the F1 score for selection. The test set was not used during tuning. SVM was not tuned because it was the slowest and weakest model.

Table 10: Best hyperparameters found

Model	Method	Best Parameters	CV F1	Time (s)
Decision Tree	GridSearchCV	max_depth = 8, min_samples_leaf = 5, class_weight = None	0.8143	19.9
Random Forest	RandomizedSearchCV	n_estimators = 200, max_depth = 15, min_samples_leaf = 1, max_features = 0.5, class_weight = None	0.8238	188.4
Gradient Boosting	RandomizedSearchCV	n_estimators = 200, max_depth = 5, learning_rate = 0.1, subsample = 0.8, min_samples_leaf = 1	0.8311	227.0
XGBoost	RandomizedSearchCV	n_estimators = 200, max_depth = 5, learning_rate = 0.1, subsample = 0.85, colsample_bytree = 0.7, scale_pos_weight = 1	0.8314	44.9

The Decision Tree chose a depth of 8 instead of 33, which removes its overfitting. Gradient Boosting and XGBoost both chose 200 trees with a depth of 5 and a learning rate of 0.1. All searches chose no class weighting: the precision is already very high (95% to 98%), so giving more weight to defaults would raise recall but lower precision more than it helps.

Table 11: Test-set metrics before and after tuning (before → after)

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Gradient Boosting	0.9310 → 0.9405	0.9500 → 0.9751	0.7228 → 0.7468	0.8210 → 0.8458	0.9325 → 0.9514
XGBoost	0.9397 → 0.9389	0.9849 → 0.9758	0.7355 → 0.7391	0.8421 → 0.8411	0.9499 → 0.9511
Random Forest	0.9352 → 0.9371	0.9699 → 0.9819	0.7264 → 0.7257	0.8306 → 0.8345	0.9367 → 0.9392
Decision Tree	0.8936 → 0.9347	0.7398 → 0.9752	0.7920 → 0.7200	0.7650 → 0.8284	0.8570 → 0.9183

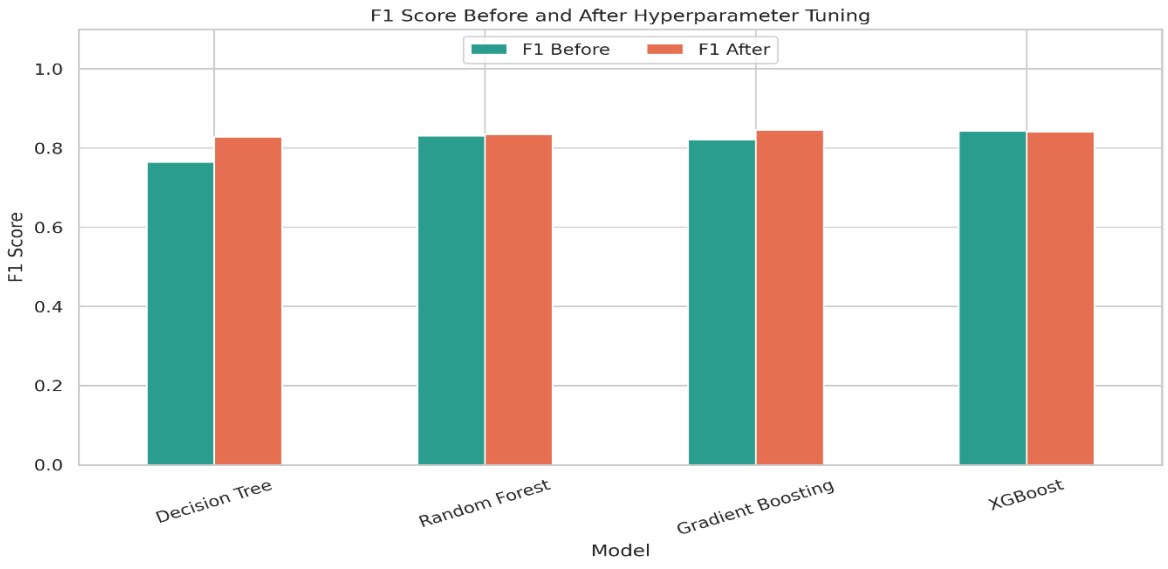


Figure 9: F1 score before and after hyperparameter tuning

Tuning improved the F1 score of three models: Decision Tree +0.0634, Gradient Boosting +0.0248 and Random Forest +0.0039. XGBoost stayed practically the same (0.8421 to 0.8411) because its

default settings were already well chosen; this difference is smaller than the variation seen in cross-validation. The Decision Tree benefited most: limiting its depth raised its accuracy from 89.36% to 93.47% and its precision from 73.98% to 97.52%. Gradient Boosting became the best tuned model (F1 0.8458, ROC-AUC 0.9514), while recall stays between 72% and 75% for all models.

7.3 Feature Importance Analysis

Two kinds of feature importance were calculated for the best tuned model (Gradient Boosting). The model importance shows how much each encoded feature helped the trees during training. The permutation importance shuffles each original column of the test set in turn and measures the drop in F1 score; it is easier to interpret because it works on the raw columns and on data the model has not seen.

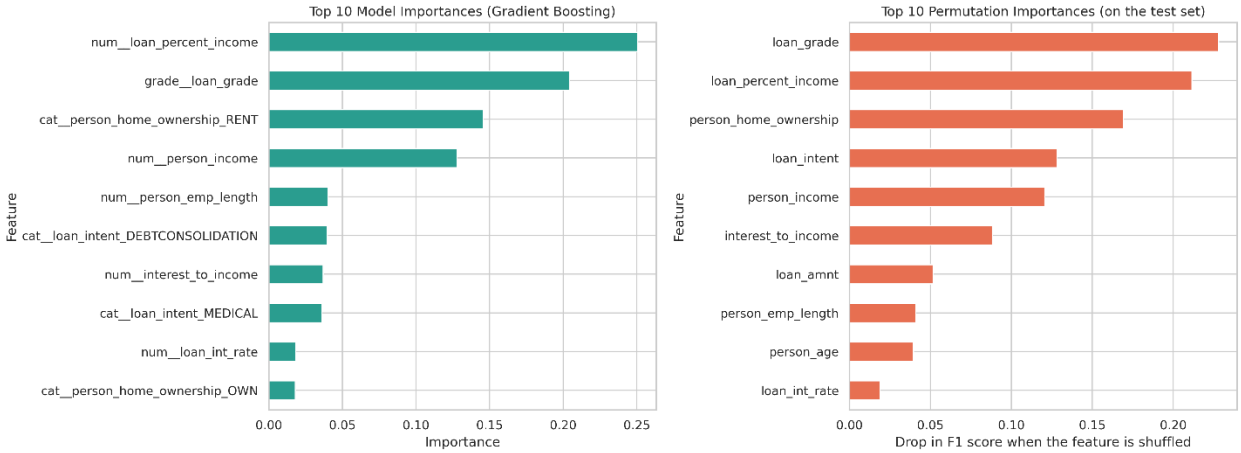


Figure 10: Model importance and permutation importance of the best model

Table 12: Top 10 columns by permutation importance (drop in F1 score)

Column	Drop in F1 Score
loan_grade	0.2280
loan_percent_income	0.2115
person_home_ownership	0.1693
loan_intent	0.1282
person_income	0.1207
interest_to_income (engineered)	0.0883
loan_amnt	0.0517
person_emp_length	0.0410
person_age	0.0396
loan_int_rate	0.0190

A loan is judged mainly by its risk grade, by how large it is compared with the income, and by the borrower's housing situation and loan purpose, which agrees with common credit-risk practice.

The engineered feature interest_to_income is in the top six, ahead of the loan amount, employment length and age. loan_int_rate has a low permutation importance although it is closely related to the loan grade: when two columns carry almost the same information, shuffling one of them costs little because the model can use the other one.

7.4 Decision Threshold Analysis

The model predicts a probability of default, and by default a loan is flagged when this probability is at least 0.5. A lower threshold flags more loans: recall rises but precision falls. The threshold was evaluated on cross-validated predictions of the training data and checked on the test set, so the test set was not used to choose it.

Table 13: Effect of the decision threshold (test set)

Threshold	Precision	Recall	F1 Score	Validation F1
0.50	0.9751	0.7468	0.8458	0.8311
0.40	0.9466	0.7630	0.8450	0.8307
0.35	0.9198	0.7687	0.8375	0.8271
0.30	0.8868	0.7842	0.8323	0.8233
0.25	0.8485	0.8096	0.8286	0.8115
0.20	0.7779	0.8272	0.8018	0.7918

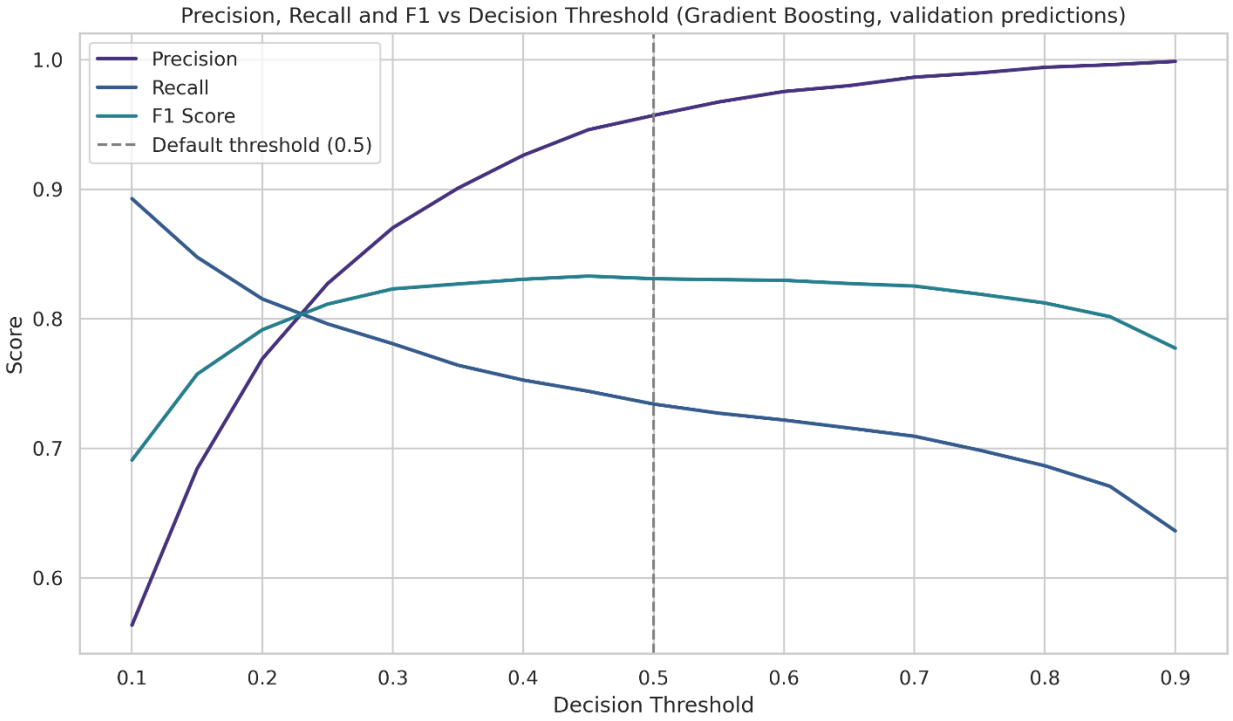


Figure 11: Precision, recall and F1 score against the decision threshold

The F1 curve is almost flat between thresholds of about 0.35 and 0.7, and the difference between 0.5 and 0.4 is below 0.001, so 0.5 was kept as the final threshold. If a missed default is much more expensive than a rejected good loan, a lower threshold of 0.3 to 0.4 can be chosen: a threshold of 0.3 finds 78.42% of the defaults with a precision of 88.68%. The Streamlit interface lets the user change the threshold.

SECTION 8: Saving the Best Model

The best tuned model (Gradient Boosting) was saved in two formats (Table 14). Joblib is efficient for objects with large NumPy arrays, which is typical for scikit-learn models, and Pickle is the standard Python serialization format. The saved object is the complete pipeline (imputation, encoding, scaling, feature selection and model), so it accepts raw applicant data and needs no separate preprocessing code.

Table 14: Saved model files

File	Format	Size	Content
credit_risk_pipeline.joblib	Joblib	858.6 KB	Complete pipeline
credit_risk_pipeline.pkl	Pickle	869.6 KB	Complete pipeline
model_info.json	JSON	small	Model name, best parameters, threshold, test metrics, input columns, library versions

Both files were loaded again and used to predict the test set. The predictions are identical to those of the original model, and the loaded model reproduces the test metrics exactly (accuracy 0.9405, recall 0.7468, F1 0.8458). The models were created with scikit-learn 1.6.1, and the same version should be used when loading them.



Figure 12: Saving and reloading the model with Joblib and Pickle

Two example applicants were scored with the loaded pipeline (Table 15). The first applicant has a grade A loan of 5,000, which is 6% of an income of 90,000, and no previous default. The second applicant has a grade E loan of 12,000, which is 48% of an income of 25,000, and a previous default. The predictions agree with common sense, and the pipeline works on raw input data.

Table 15: Predictions for two example applicants

Applicant	Loan Grade	Loan / Income	Default Probability	Decision
Applicant 1	A	6%	1.33%	Low risk
Applicant 2	E	48%	99.85%	High risk

```
])

new_X = add_engineered_features(new_customers)
prob = model_joblib.predict_proba(new_X)[:, 1]
new_customers['default_probability'] = prob.round(4)
new_customers['decision'] = np.where(prob >= FINAL_THRESHOLD, 'High risk (default likely)', 'Low risk')
display(new_customers[['person_income', 'loan_grade', 'loan_amnt', 'loan_percent_income',
                       'default_probability', 'decision']])
```

	person_income	loan_grade	loan_amnt	loan_percent_income	default_probability	decision
0	90000	A	5000	0.06	0.0133	Low risk
1	25000	E	12000	0.48	0.9985	High risk (default likely)

Figure 13: Predictions of the saved pipeline for new applicants

SECTION 9: Prediction Interface with Streamlit (Bonus)

A simple interface was built with Streamlit (app/app.py). It loads the saved Joblib pipeline, calculates the engineered features automatically and shows the default probability of a loan applicant. It contains:

- A form with the 11 inputs (age, income, home ownership, employment length, credit history, previous default, loan purpose, loan grade, loan amount and interest rate); the loan as a share of income is calculated automatically.
- The default probability with a progress bar and a clear result: high risk or low risk.
- A threshold slider in the sidebar, based on the analysis of Section 7.4.
- A Model Information tab with the test metrics, the best hyperparameters and the most important features.
- A Batch Predictions tab, where a CSV file with many applicants can be uploaded and the results downloaded.

The interface was tested with the two example applicants of Table 15 and returns the same results as the notebook.

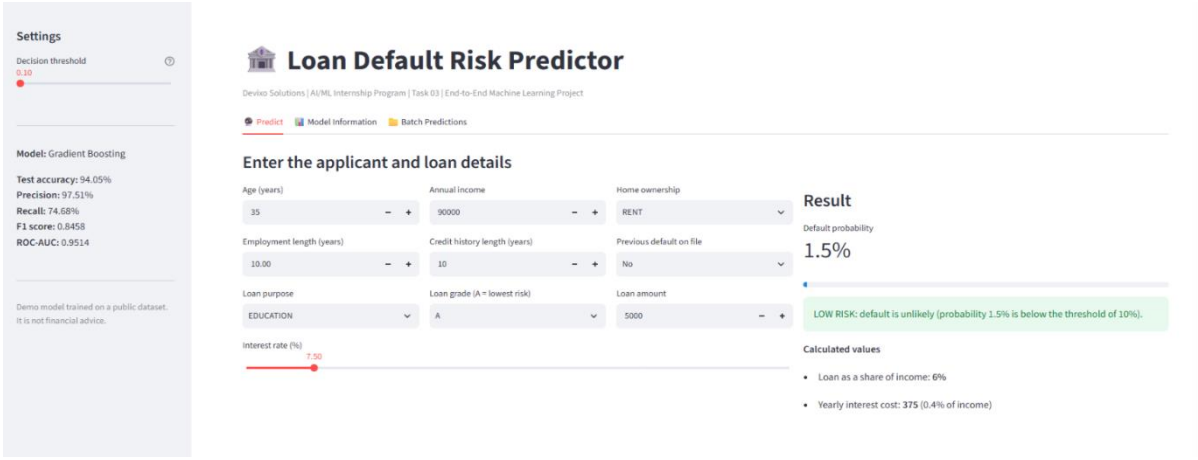


Figure 14: Prediction interface: low-risk applicant

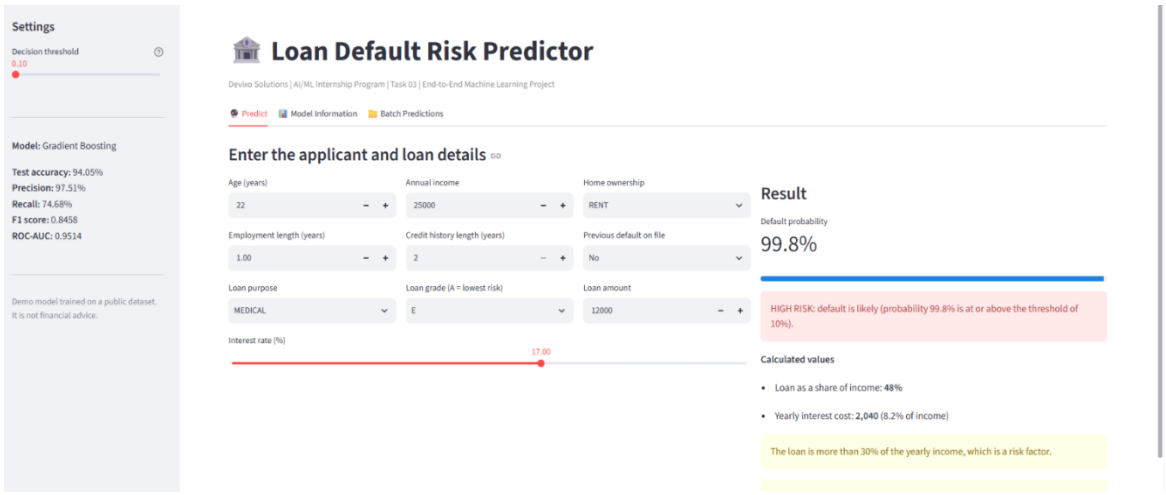


Figure 15: Prediction interface: high-risk applicant

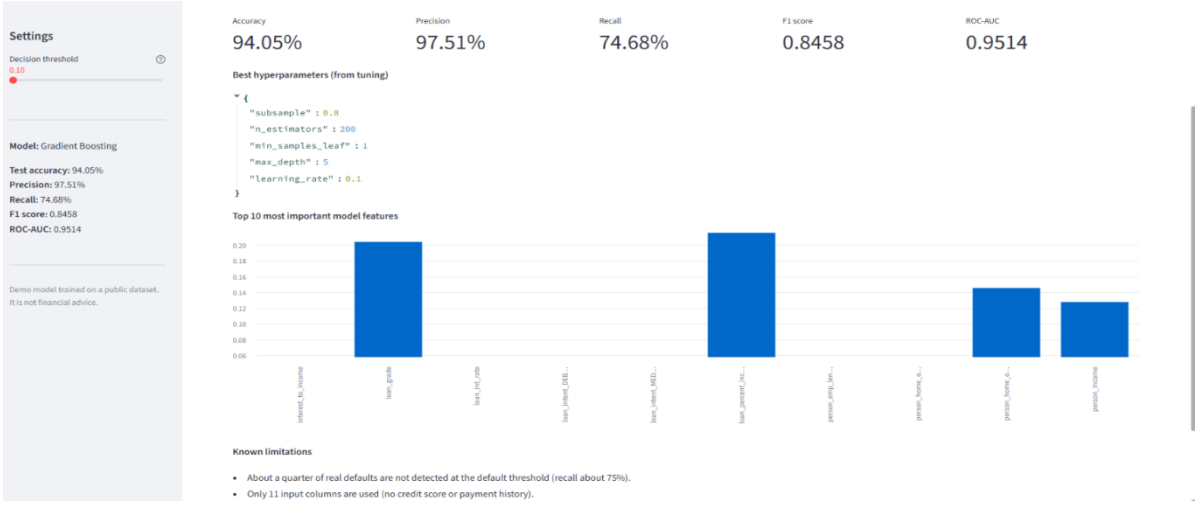


Figure 16: Model information tab

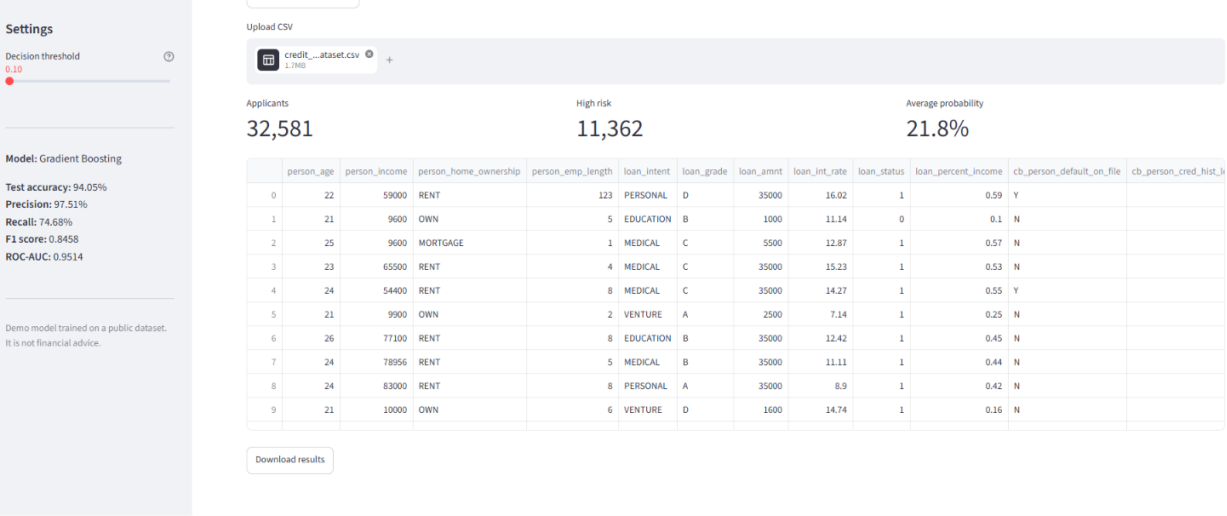


Figure 17: Batch predictions tab

Live app: https://task03ai-mlloandefaultprediction.streamlit.app/

SECTION 10: Model Selection and Results

Table 16 shows the final results of the tuned models on the test set, together with the best cross-validation F1 score and the time needed for tuning.

Table 16: Final comparison of the models (test set)

Model	Tuning (s)	CV F1	Accuracy	Precision	Recall	F1	ROC-AUC
Gradient Boosting	227.0	0.8311	0.9405	0.9751	0.7468	0.8458	0.9514
XGBoost	44.9	0.8314	0.9389	0.9758	0.7391	0.8411	0.9511
Random Forest	188.4	0.8238	0.9371	0.9819	0.7257	0.8345	0.9392
Decision Tree	19.9	0.8143	0.9347	0.9752	0.7200	0.8284	0.9183
SVM (not tuned)	-	0.7562	0.9101	0.9196	0.6453	0.7584	0.8937

Gradient Boosting was selected as the final model. After tuning it has the highest F1 score (0.8458), accuracy (94.05%) and recall (74.68%) on the test set, and a ROC-AUC of 0.9514. XGBoost is almost identical (F1 0.8411, ROC-AUC 0.9511, cross-validation F1 0.8314 against 0.8311) and was about five times faster to tune, so the difference between the two lies within the variation of cross-validation, and XGBoost is an equally good choice when speed matters. Random Forest (F1 0.8345) and the tuned Decision Tree (F1 0.8284) are weaker, and the untuned SVM was the slowest and weakest model. The boosting models are also stable (cross-validation standard deviation of about 0.004).

On the test set of 6,482 loans, the final model reaches an accuracy of 94.05%, a precision of 97.51%, a recall of 74.68%, an F1 score of 0.8458 and a ROC-AUC of 0.9514, compared with 78.18% accuracy for always predicting "no default". With a threshold of 0.5 it finds about 1,059 of the 1,418 real defaults and wrongly flags only about 27 good loans. The most important inputs

are the loan grade, the loan amount as a percentage of income, home ownership, the loan purpose and the income.

SECTION 11: Limitations and Future Improvements

11.1 Limitations

- Recall: about a quarter of the real defaults (about 359 of 1,418) are not detected at the default threshold. Lowering the threshold finds more defaults but increases the false alarms.
- Data: only 11 input columns are available (no credit score, existing debts or payment history), the borrowers are mostly young (median age 26) and the dataset is a public sample, so the model may not transfer to another lender or country.
- Loan grade and interest rate are assigned by the lender after a risk assessment, so the model can only be used once the grade is known, and these columns may carry information that is close to the outcome.
- Evaluation: one fixed train-test split and cross-validation were used, but no time-based validation, so the performance over time is unknown.
- Fairness: features such as age and home ownership were not checked for unfair bias.
- Tuning: the search spaces were limited to keep the training time reasonable, and SVM was not tuned.

11.2 Future Improvements

- Add more informative features (credit score, debt-to-income ratio, payment history) and test the model on other credit datasets.
- Choose the decision threshold from the real costs of a missed default and of a rejected good loan.
- Calibrate the predicted probabilities and explain single decisions with SHAP values.
- Check the model for fairness across age and other groups, and validate it on a time-based split.
- Try other techniques for imbalanced data (for example SMOTE), a wider hyperparameter search and model stacking.
- Deploy the pipeline as an API, monitor its performance on new loans and retrain it regularly.

SECTION 12: Conclusion

In this task, a complete machine learning project was built on the Credit Risk dataset (32,581 loan applicants, 21.8% defaults). The data was cleaned (165 duplicates and 7 impossible rows removed), four features were engineered, and a reusable pipeline handled missing values,

encoding, scaling and feature selection. Five models were trained and compared. With default settings XGBoost was the best (F1 0.8421), the Decision Tree and the Random Forest overfitted, and every model found only about 65% to 79% of the defaults.

Cross-validation, hyperparameter tuning and feature importance analysis improved and explained the models. The tuned Gradient Boosting model is the final choice: accuracy 94.05%, precision 97.51%, recall 74.68%, F1 score 0.8458 and ROC-AUC 0.9514. It depends mainly on the loan grade, the loan amount relative to the income and the housing situation, and the engineered feature interest_to_income is among the most important ones. The complete pipeline was saved with Joblib and Pickle, verified by reloading it, and used in a Streamlit interface that gives a default probability for a new applicant. The main limitation is that about a quarter of the real defaults are still missed, which future work can address with more informative features and a threshold based on real costs. Through this task I learned how to build a leak-free pipeline, how to compare and tune models fairly, and how to save and deploy a model so that it can be reused.

SECTION 13: GitHub Repository Link

Table 17: Project links

Item	Link
GitHub Repository	https://github.com/Amankhan987/Task_03_AI-ML_Loan_Default_Prediction
Live Streamlit App	https://task03ai-mlloandefaultprediction.streamlit.app/
Dataset Source (Kaggle)	https://www.kaggle.com/datasets/laotse/credit-risk-dataset

The repository contains the Jupyter notebook, the dataset, the saved models (Joblib and Pickle), the Streamlit application, all charts and result tables, requirements.txt and README.md.